Applied Generative AI & LLM Systems

Agentic Systems

2026.03.26

AGENDA

1. Introduction
2. The **anatomy** of agentic systems
3. Tool (Re)use
4. Context Management

1

Agentic Systems

Introduction

2017: Machine Translation: Attention Is All You Need

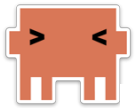
2019: Decoder Only Generative Model. GPT2

2020-2022: RAG & RLHF - GPT3.5

2023/24: Multimodality (GPT-4V, Speech2Speech Real Time API)

Early 2025: Reasoning models, RLVR

Mid-Late 2025: agentic coding, OpenClaw

Now 

INTRODUCTION | A QUICK TIMELINE TO NOW

2017: Machine Translation: Attention Is All You Need

2019: Decoder Only Generative Model. GPT2

2020-2022: RAG & RLHF - GPT3.5

2023/24: Multimodality (GPT-4V, Speech2Speech Real Time API)

JustAnotherClaw

Early 2025: Reasoning models, RLVR

Mid-Late 2025: agentic coding, OpenClaw

Now

TRADITIONAL CHATBOT vs. AGENTIC SYSTEMS

JustAnotherChatbot

Traditional Chatbot

Text in Text out, without side effects

VS

JustAnotherClaw

Agentic System

Text in Text out, in a loop[1] and with side effects

1. We use the term "loop" as simplification for any "Multi-step" pattern where the model is called more than once as part of a single task. The pattern of those steps (sequential, parallel, iterative, branching) is an implementation detail.

TRADITIONAL CHATBOT vs. AGENTIC SYSTEMS

Give me recipe for pancakes?

JustAnotherChatbot

Here's a simple, reliable recipe for classic fluffy pancakes you can make at home:

🥞 **Easy Pancake Recipe Ingredients**

- 1 cup (125 g) all-purpose flour
- 2 tablespoons sugar

[...]

VS

JustAnotherClaw

Sure, let me check what you have in your fridge

calling Fridge API

Got it! Let's search the internet for some inspiration

WebSearch(„Pancake Recipes with milk, [...]")

Great! I'll write my top 3 picks into a markdown file.

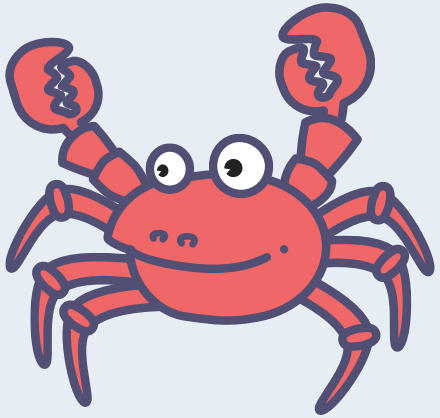
Write(./pancake-Recipes.md)

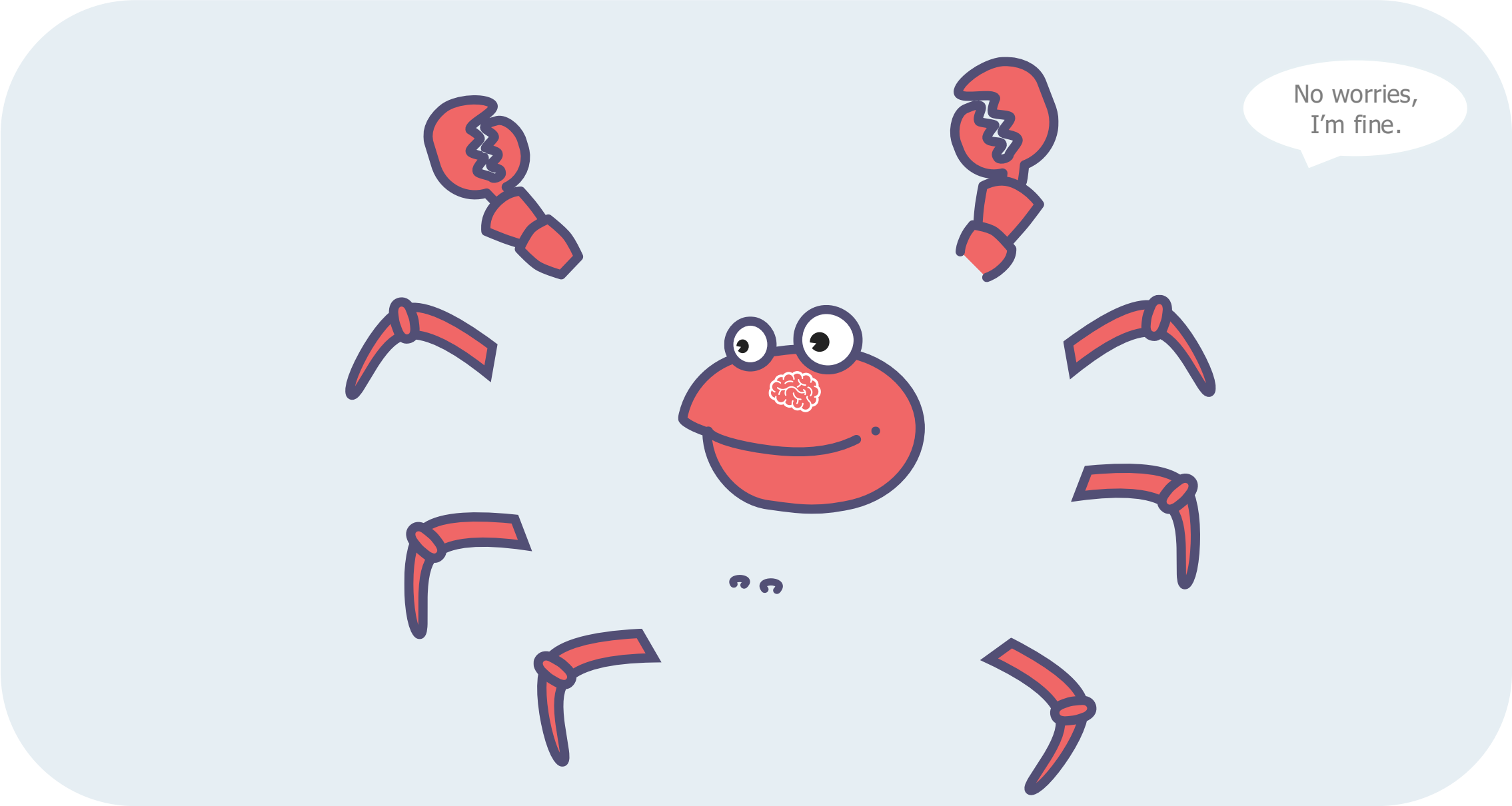
Want me to give you step by step instructions for any of the recipes?

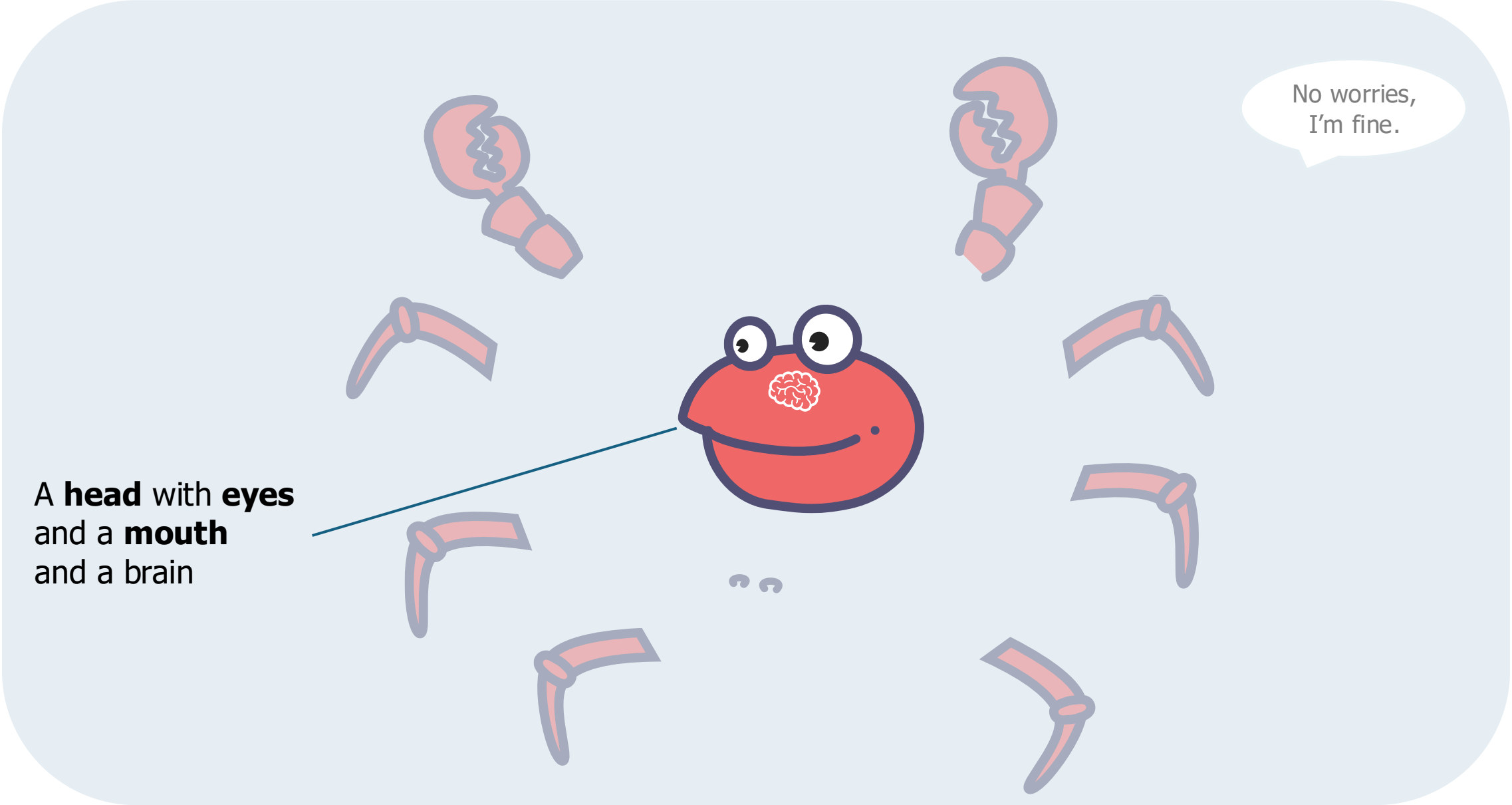
2

Agentic Systems

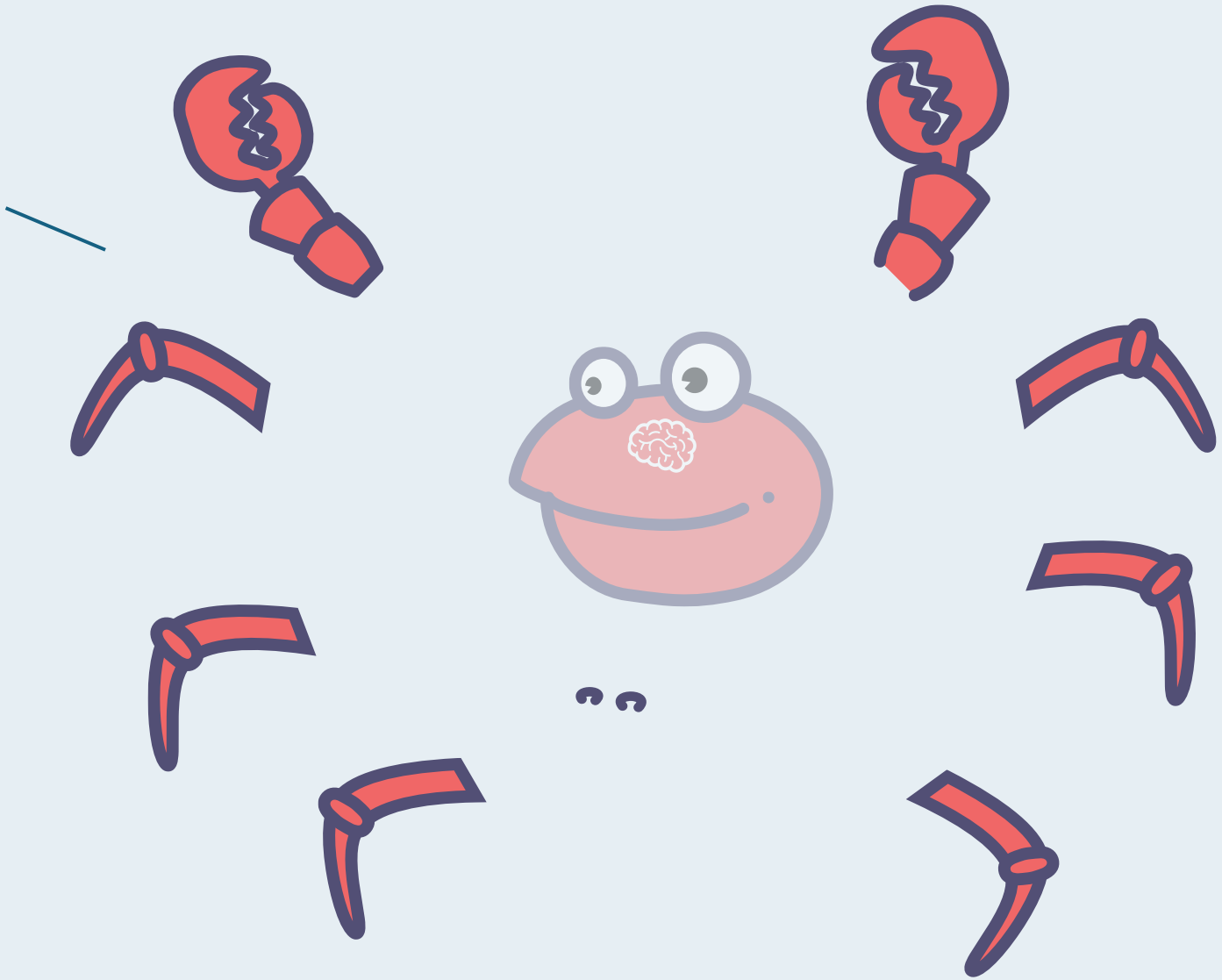
Anatomy

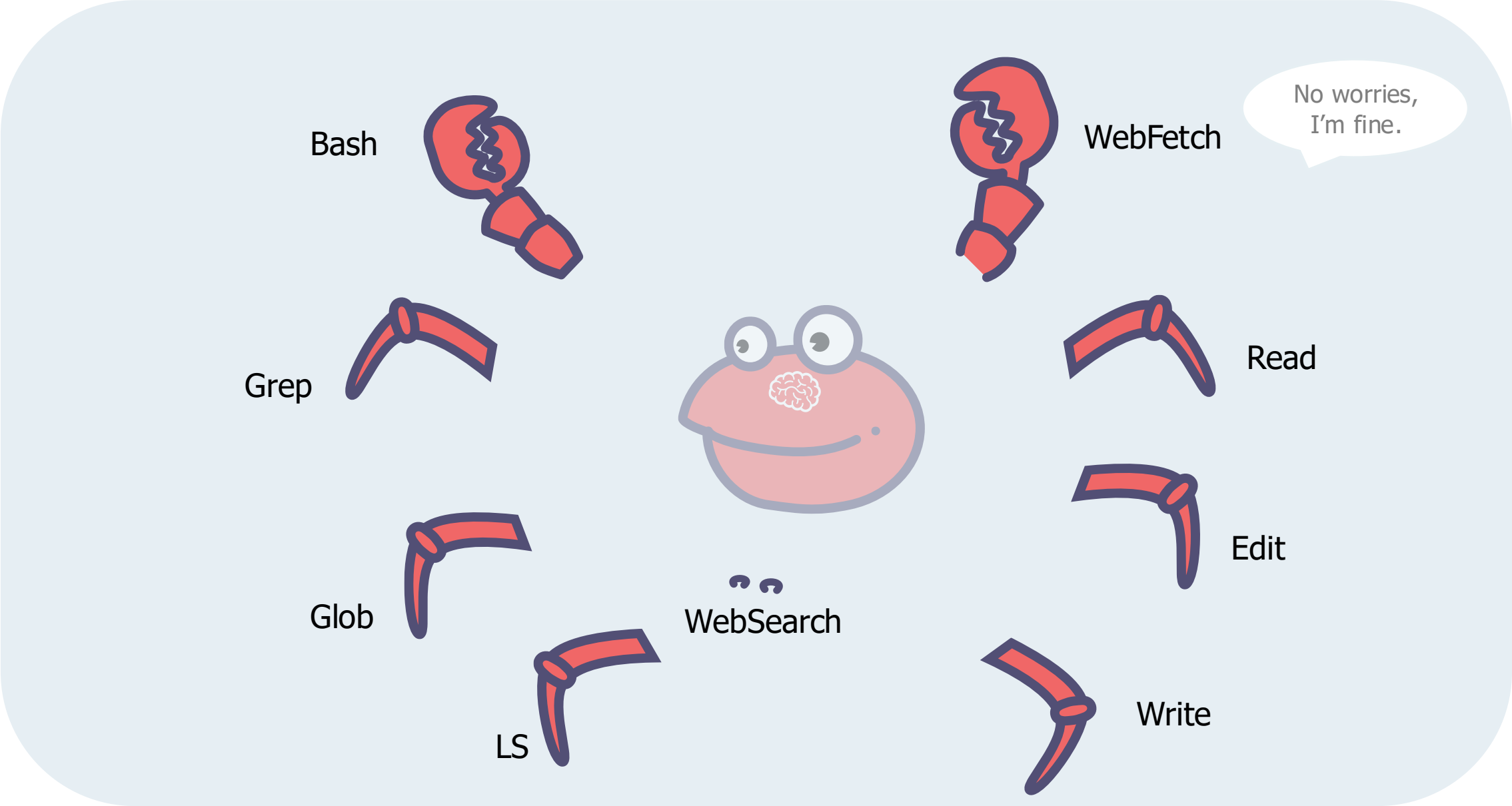


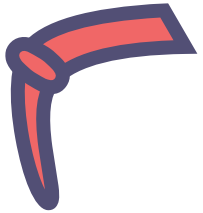

No worries,
I'm fine.


No worries,
I'm fine.
A **head** with **eyes**
and a **mouth**
and a brain

Tools the brain can *"use"* to act in the world

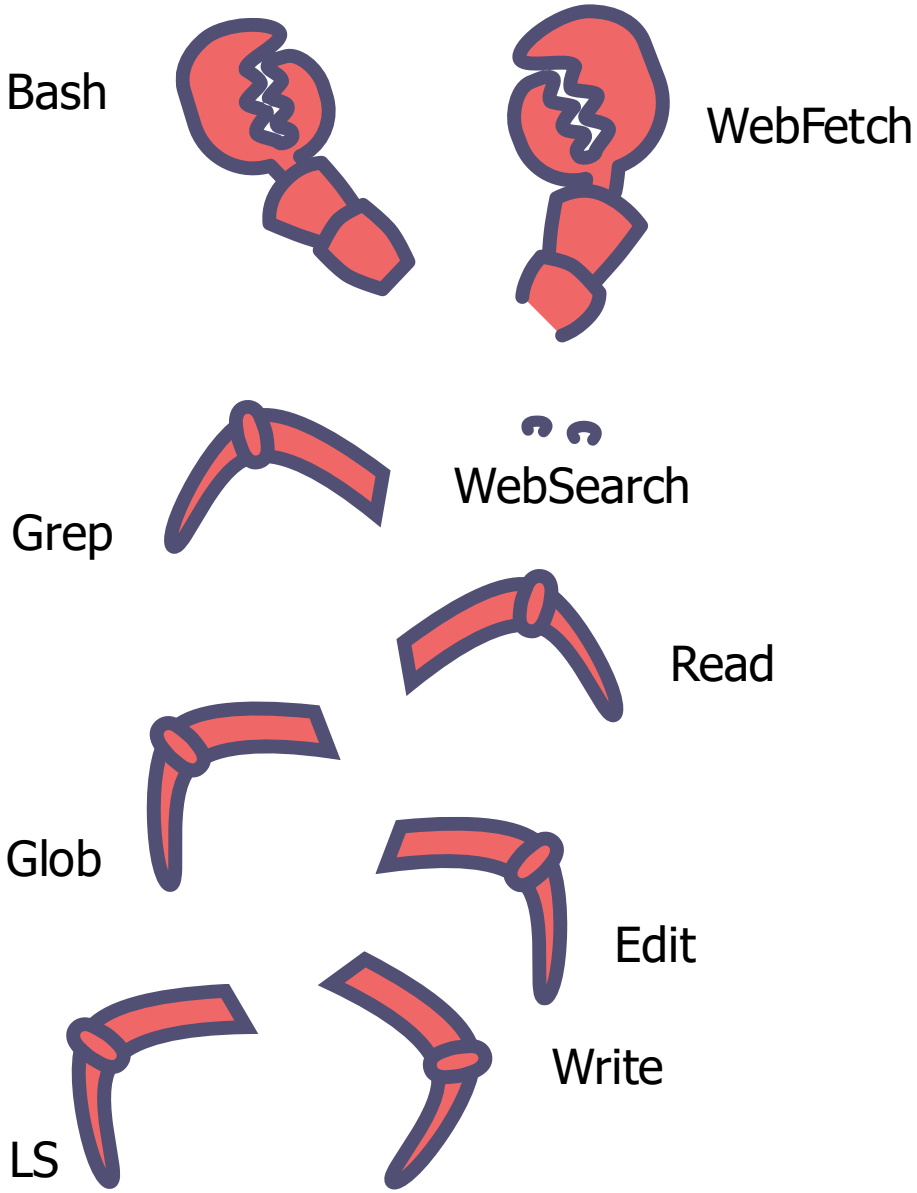



Bash
WebFetch
No worries,
I'm fine.
Grep
Read
Edit
Glob
WebSearch
LS
Write



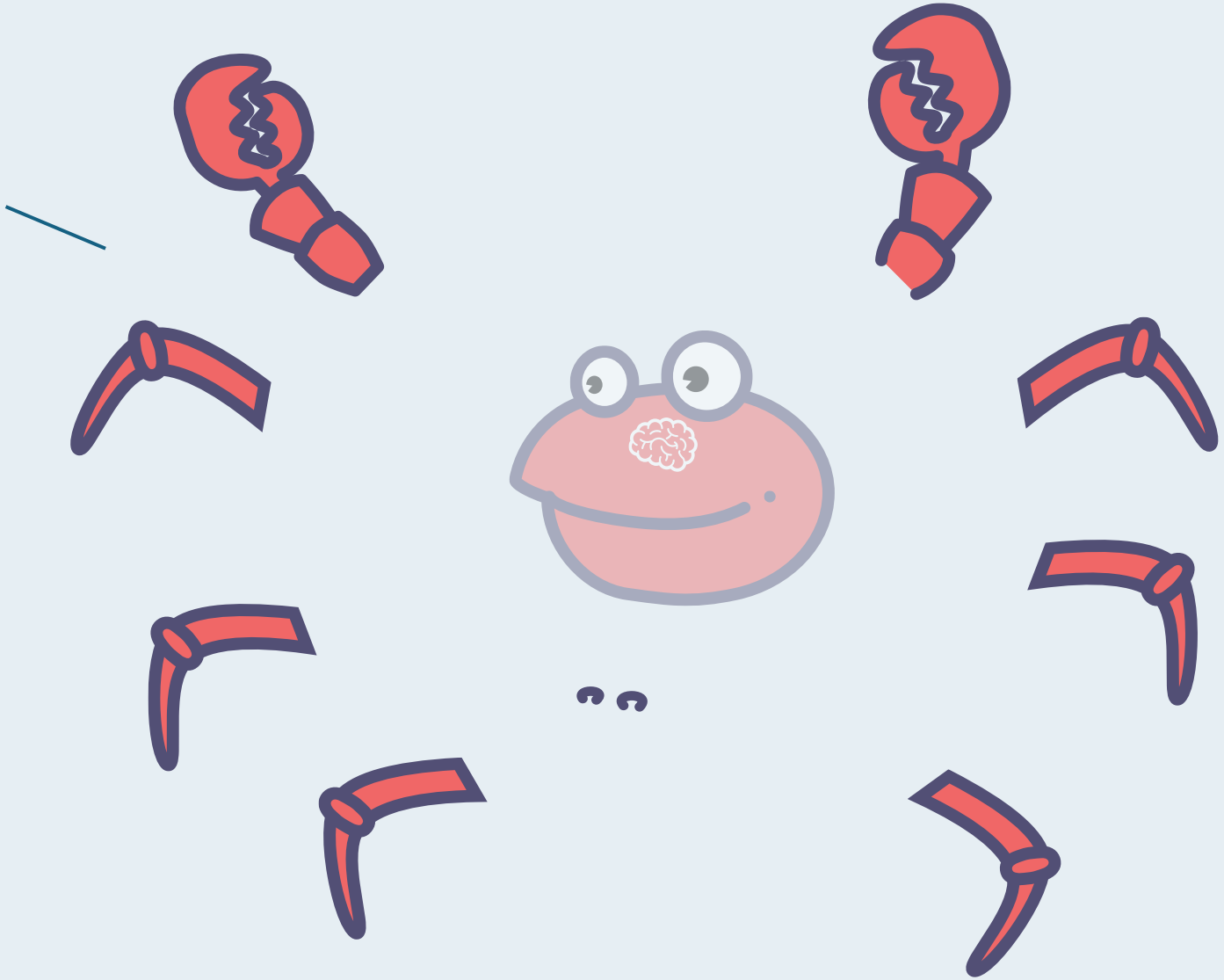

```python
import os

@tool(
    name="list_files",
    description="List files in a directory.",
)
def list_files_tool(path: str = ".", recursive: bool = False) -> dict:
    results = []

    if recursive:
        for root, dirs, files in os.walk(path):
            for name in files:
                full = os.path.join(root, name)
                results.append({"path": full})

    else:
        for name in os.listdir(path):
            full = os.path.join(path, name)
            if os.path.isfile(full):
                results.append({"path": full})
    return {"files": results}
```

tools/ls.py

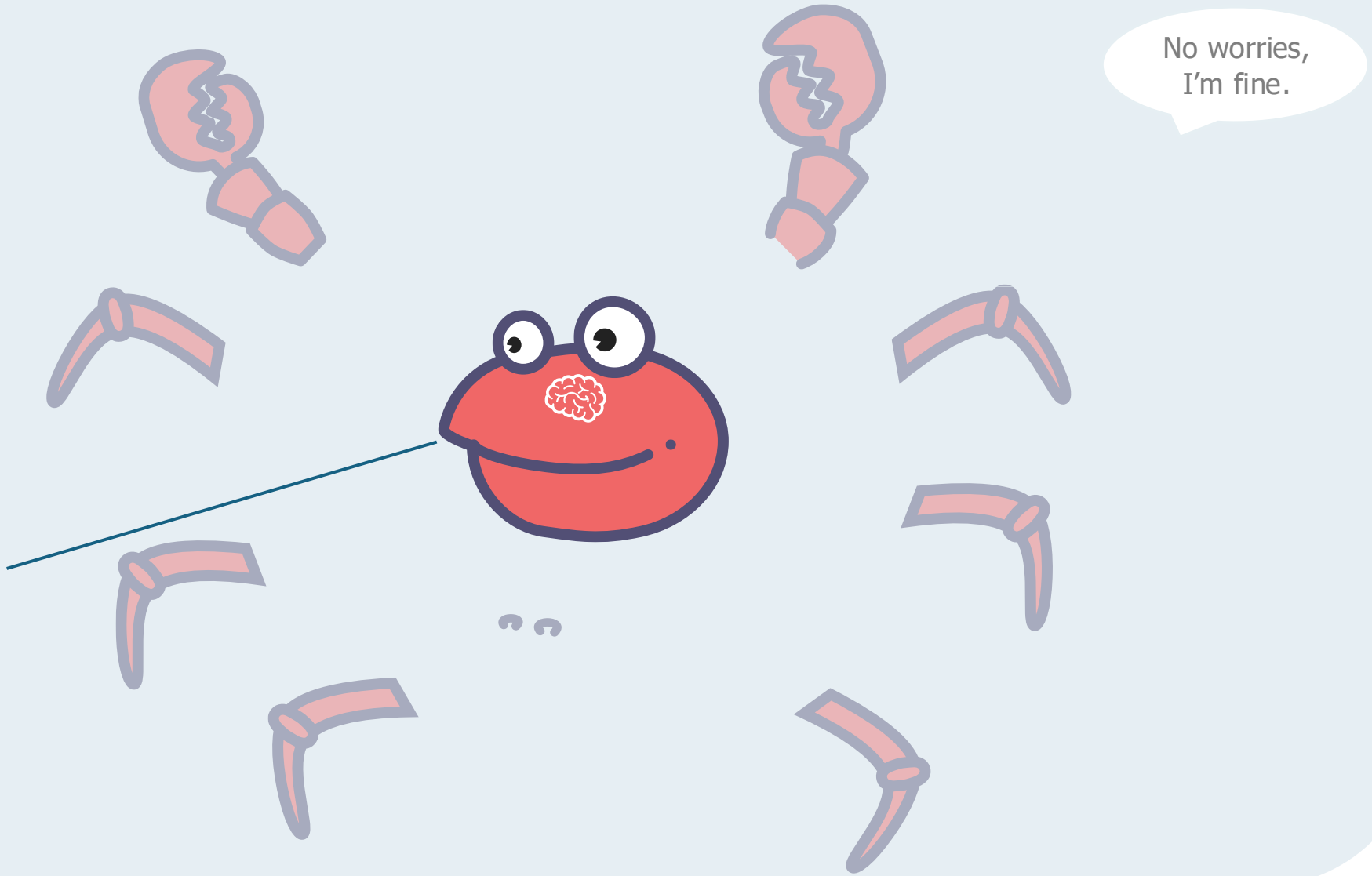



```
tools/
├─ bash.py
├─ edit.py
├─ glob.py
├─ grep.py
├─ ls.py
├─ read.py
├─ web_fetch.py
├─ web_search.py
└─ write.py
```

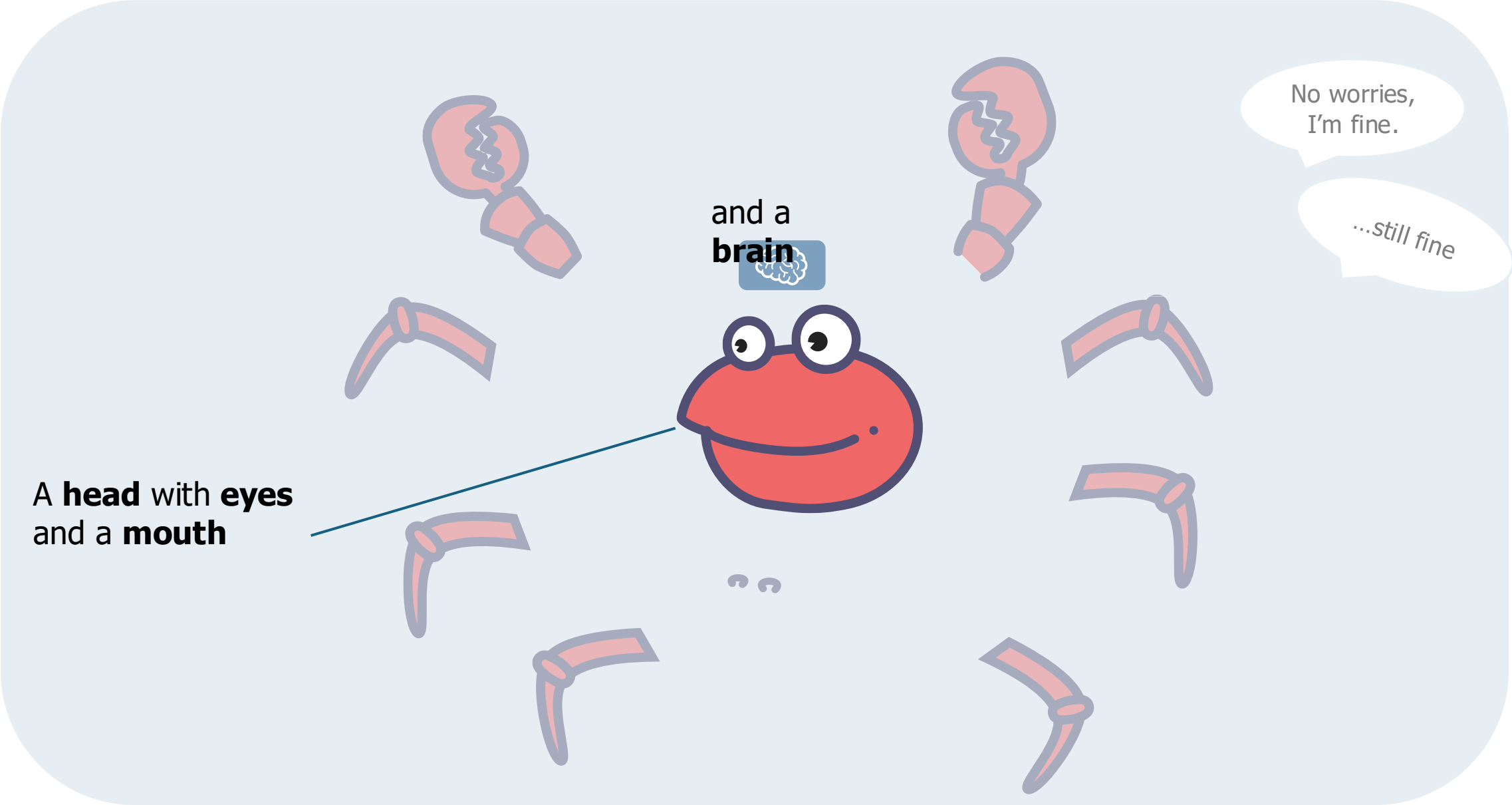
Tools the brain can *"use"* to act in the world

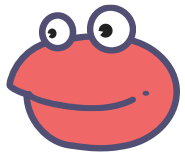


No worries, I'm fine.



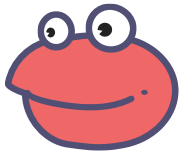
A **head** with **eyes**
and a **mouth**
and a
brain


No worries,
I'm fine.
...still fine
and a
brain
A **head** with **eyes**
and a **mouth**



A **head** with **eyes and** a **mouth**

1. Connects / “harnesses” the brain and tools
2. Runs the core “agent loop” (while True)

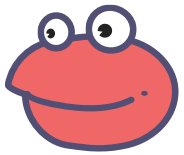


A **head** with **eyes and** a **mouth**

1. Connects / “harnesses” the brain and tools

JSON-Schema

```
{
  "name": "ls",
  "description": "List files and directories at a given path",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": { "type": "string", "description": "The directory path to list" },
      "recursive": { "type": "boolean", "description": "Whether to list files recursively” }
    },
    "required": ["path"]
  }
}
```



A **head** with **eyes and** a **mouth**

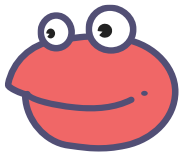
1. Connects / “harnesses” the brain and tools

JSON-Schema

```
{
  "name": "ls",
  "description": "List files and directories at a given path",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": { "type": "string", "description": "The directory path to list" },
      "recursive": { "type": "boolean", "description": "Whether to list files recursively" }
    },
    "required": ["path"]
  }
}
```

Tool Use Request

```
{
  "type": "tool_use",
  "id": "toolu_abc123",
  "name": "ls",
  "input": {
    "path": ".",
    "recursive": false
  }
}
```



A **head** with **eyes and** a **mouth**

1. Connects / “harnesses” the brain and tools

JSON-Schema

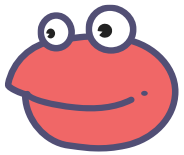
```
{
  "name": "ls",
  "description": "List files and directories at a given path",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": { "type": "string", "description": "The directory path to list" },
      "recursive": { "type": "boolean", "description": "Whether to list files recursively" }
    },
    "required": ["path"]
  }
}
```

Tool Use Request

```
{
    "type": "tool_use",
    "id": "toolu_abc123",
    "name": "ls",
    "input": {
      "path": ".",
      "recursive": false
    }
}
```

Code Execution

```
{
    "type": "tool_result",
    "tool_use_id": "toolu_abc123",
    "content": ”pancake-Recipes.md”
}
```



A **head** with **eyes and** a **mouth**

1. Connects / “harnesses” the brain and tools

JSON-Schema

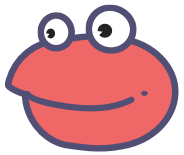
```
{
  "name": "ls",
  "description": "List files and directories at a given path",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": { "type": "string", "description": "The directory path to list" },
      "recursive": { "type": "boolean", "description": "Whether to list files recursively" }
    },
    "required": ["path"]
  }
}
```

Tool Use Request

```
{
    "type": "tool_use",
    "id": "toolu_abc123",
    "name": "ls",
    "input": {
        "path": ".",
        "recursive": false
    }
}
```

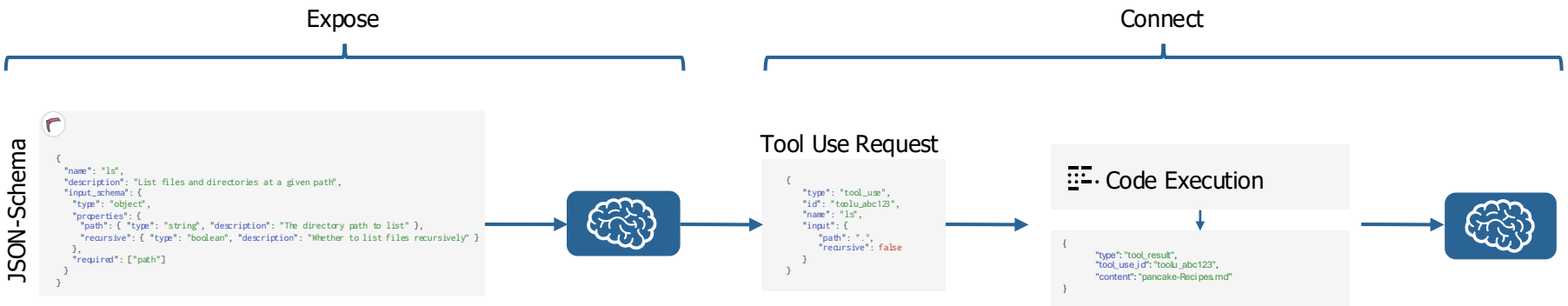
Code Execution

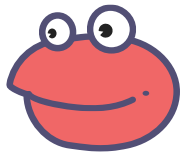
```
{
        "type": "tool_result",
        "tool_use_id": "toolu_abc123",
        "content": "pancake-Recipes.md"
}
```



A **head** with **eyes and** a **mouth**

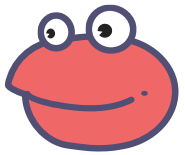
1. Connects / “harnesses” the brain and tools





A **head** with **eyes and** a **mouth**

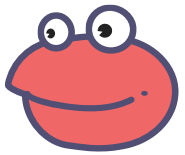
1. Connects / “harnesses” the brain and tools
2. Runs the core “agent loop” (while True)



A **head** with **eyes and** a **mouth**

2. Runs the core “agent loop” (while True)

```
while True:
    response = call_brain(messages)

    if response.stop_reason == "end_turn":      # brain said <|eot_id|>
        break                                   # done, show to user

    if response.stop_reason == "tool_use":      # brain said <|eom_id|>
        result = execute_tool(response)         # run tool
        messages.append(result)                 # add result to history
        continue
```



A **head** with **eyes and** a **mouth**

2. Runs the core "agent loop" (while True)

```
while True:
    response = call_brain(messages)

    if response.stop_reason == "end_turn":      # brain said <|eot_id|>
        break                                   # done, show to user

    if response.stop_reason == "tool_use":      # brain said <|eom_id|>
        result = execute_tool(response)         # run tool
        messages.append(result)                 # add result to history
        continue
```

A **head** with **eyes and** a
uth

"agent loop" (while

Obviously it's not "while true"
in some high level language?

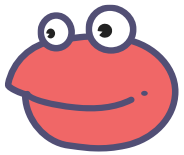
```
while True:
    response = call_brain(messages)

    if response.stop_reason == "end_turn":      # brain said <|eot_id|>
        break                                   # done, show to user

    if response.stop_reason == "tool_use":      # brain said <|eom_id|>
        result = execute_tool(response)         # run tool
        messages.append(result)                 # add result to history
        continue
```

AGENTIC SYSTEMS | JUSTANOTHERCLAW > HARNESS

```
while (true) {
   let hasMoreToolCalls = true;

   // Inner loop: process tool calls and steering messages
   while (hasMoreToolCalls || pendingMessages.length > 0) {
      if (!firstTurn) {
         await emit({ type: "turn_start" });
      } else {
         firstTurn = false;
      }

      // Process pending messages (inject before next assistant response)
      if (pendingMessages.length > 0) {
         for (const message of pendingMessages) {
            await emit({ type: "message_start", message });
            await emit({ type: "message_end", message });
            currentContext.messages.push(message);
            newMessages.push(message);
         }
         pendingMessages = [];
      }

      // Stream assistant response
      const message = await streamAssistantResponse(
              currentContext, config, signal, emit, streamFn
      );
      newMessages.push(message);

      if (message.stopReason === "error" || message.stopReason === "aborted")
      {
         await emit({ type: "turn_end", message, toolResults: [] });
         await emit({ type: "agent_end", messages: newMessages });
         return;
      }
```

```
         // Check for tool calls
         const toolCalls = message.content.filter((c) => c.type === "toolCall");
         hasMoreToolCalls = toolCalls.length > 0;

         const toolResults: ToolResultMessage[] = [];

         if (hasMoreToolCalls) {
                toolResults.push(
                ...(await executeToolCalls(currentContext, message,
                                                    config, signal, emit)
                            )
                );

             for (const result of toolResults) {
                            currentContext.messages.push(result);
                            newMessages.push(result);
                }
         }

         await emit({ type: "turn_end", message, toolResults });
         pendingMessages = (await config.getSteeringMessages?.()) || [];
      }

      // Agent would stop here. Check for follow-up messages.
      const followUpMessages = (await config.getFollowUpMessages?.()) || [];
      if (followUpMessages.length > 0) {
             // Set as pending so inner loop processes them
             pendingMessages = followUpMessages;
             continue;
      }

      break; // No more messages, exit
}
```

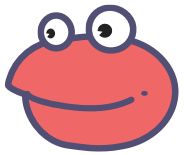
https://github.com/badlogic/pi-mono/blob/main/packages/agent./agent-loop.ts



A **head** with **eyes and** a **mouth**

2. Runs the core “agent loop” (while True)

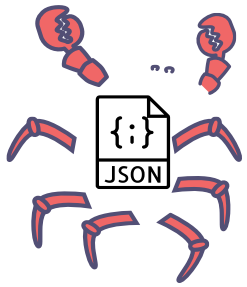
```
while True:
    response = call_brain(messages)

    if response.stop_reason == "end_turn”:
        break

    if response.stop_reason == "tool_use":
        result = execute_tool(response)
        messages.append(result)
        continue
```



A **head** with **eyes and** a **mouth**

2. Runs the core “agent loop” (while True)

```
messages = ?

while True:
      response = call_brain(messages)

      if response.stop_reason == "end_turn”:
            break

      if response.stop_reason == "tool_use":

            result = execute_tool(response)
            messages.append(result)
            continue
```



Tool Definitions

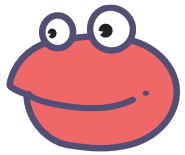
Custom instructions (e.g. SOUL.md)

System Prompt

+

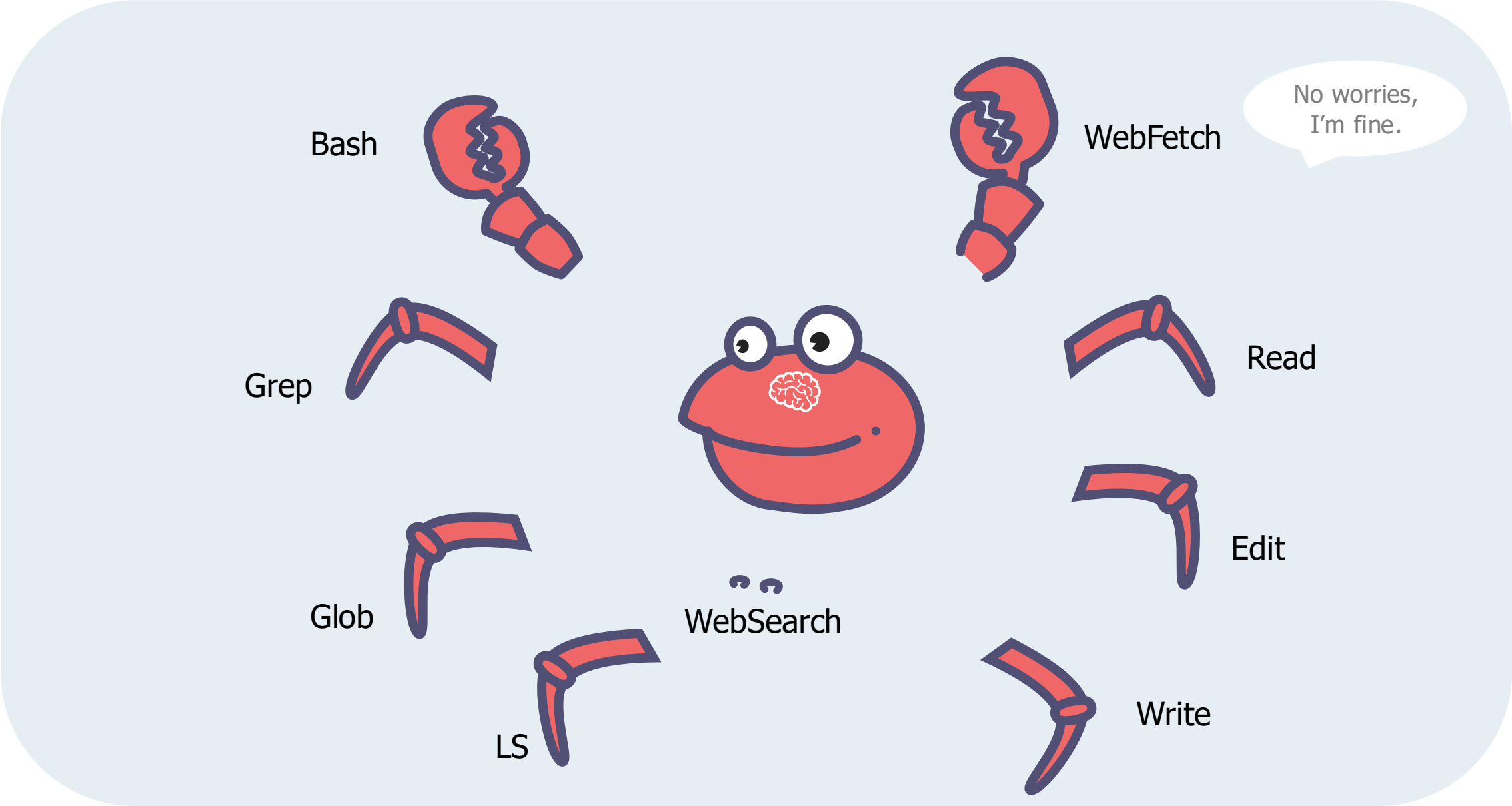
Can you check whether we have already saved the pancake recipes?

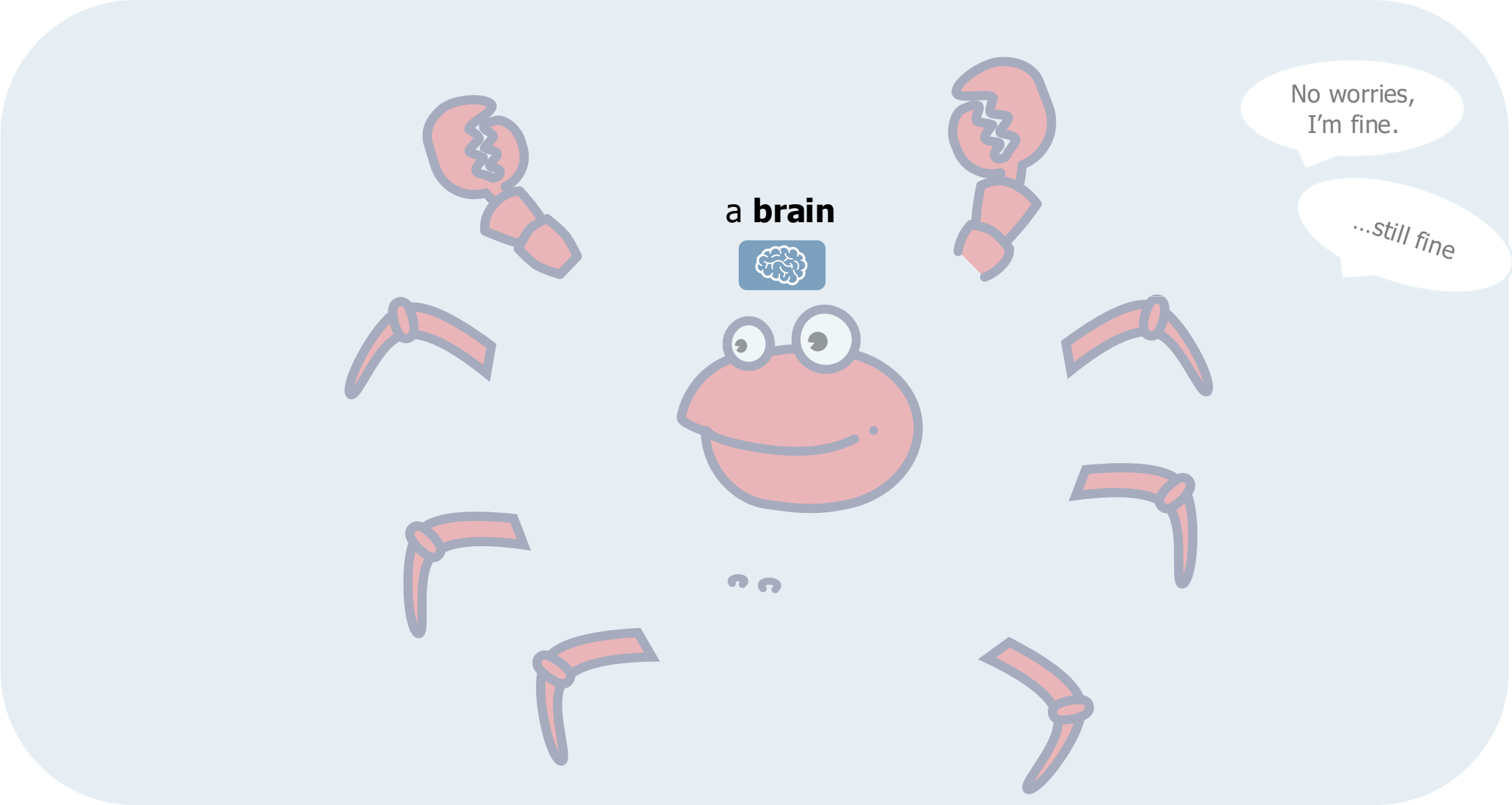
User Prompt



A **head** with **eyes and** a **mouth**

1. Connects / “harnesses” the brain and tools
2. Runs the core “agent loop” (while True)


Bash
WebFetch
No worries,
I'm fine.
Grep
Read
Edit
Glob
WebSearch
LS
Write


a **brain**
No worries, I'm fine.
...still fine

a **brain**

1. Supporting tissue
2. A set of interconnected neurons

1. Supporting tissue

```
[
  {"role": "system", "content": "…"},
  {"role": "user", "content": "…"},
]
```

→

```
parsed_input = parse_input(input)
tokens = tokenize(parsed_input)
generated = []

while True:
     probs = model.forward(tokens)
     next_token = sample(probs)
     generated.append(next_token)

     if next_token == EOM_TOKEN: # <|eom_id|>
          return parse(generated), "tool_use"
     if next_token == EOT_TOKEN: # <|eot_id|>
          return parse(generated), "end_turn"
     if len(generated) >= max_tokens:
          return parse(generated), "max_tokens"

tokens.append(next_token)
```

→

```
{
 "role": "assistant",
 "content": [
  {
   "type": "text",
   "text": "Let me check that directory."
  },
  {
   "type": "tool_use",
   "id": "toolu_abc123",
   "name": "ls",
   "input": {"path": "."}
  }
 ],
 "stop_reason": "tool_use"
}
```

1. Supporting tissue

```
parsed_input = parse_input(input)
tokens = tokenize(parsed_input)
generated = []

while True:
    probs = model.forward(tokens)
    next_token = sample(probs)
    generated.append(next_token)

    if next_token == EOM_TOKEN: # <|eom_id|>
        return parse(generated), "tool_use”
    if next_token == EOT_TOKEN: # <|eot_id|>
        return parse(generated), "end_turn”
    if len(generated) >= max_tokens:
        return parse(generated), "max_tokens"

tokens.append(next_token)
```

=

LLM inference engines

vLLM

LLaMA C++

1. Supporting tissue

```
parsed_input = parse_input(input)
. . .
```

=

```
[
  {"role": "system", "content": "You are a general assistant."},
  {"role": "user", "content": "Can you check whether we have already saved the pancake recipes?"},
  {"role": "assistant", "content": [
    {"type": "text", "text": "Let me check."},
    {"type": "tool_use", "id": "toolu_abc", "name": "ls", "input": {"path": "."}}
  ]},
  {"role": "user", "content": [
    {"type": "tool_result", "tool_use_id": "toolu_abc", "content": "pancake-Recipes.md"}
  ]}
]
```

```
parsed_input = parse_input(input)

. . .
```

=

```
[
  {"role": "system", "content": "You are a general assistant. You have access to these tools [...]"},
  {"role": "user", "content": "Can you check whether we have already saved the pancake recipes?"},
  {"role": "assistant", "content": [
    {"type": "text", "text": "Let me check."},
    {"type": "tool_use", "id": "toolu_abc", "name": "ls", "input": {"path": "."}}
  ]},
  {"role": "user", "content": [
    {"type": "tool_result", "tool_use_id": "toolu_abc", "content": "pancake-Recipes.md"}
  ]}
]
```

↓

Ultimately, the LLM still only receives a single piece of text and produces a single probability distribution based on it.

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a general assistant.
You have access to these tools:
{"name": "ls", "description": "List files", "input_schema": {...}}
<|eot_id|><|start_header_id|>user<|end_header_id|>
Can you check whether we have already saved the pancake recipes
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
Let me check.
<tool_call>
{"name": "ls", "input": {"path": ”."}}
</tool_call><|eot_id|><|start_header_id|>user<|end_header_id|>
[Tool result for toolu_abc]: pancake-Recipes.md<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Ultimately, the LLM still only receives a single piece of text and produces a single probability distribution based on it.

In the end “chat” and “conversations” and “deepresearch” etc. are nothing more than a relatively complex way of engineering a single prompt.

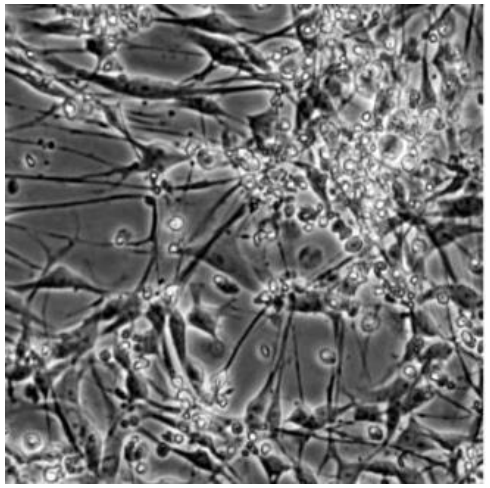
a **brain**

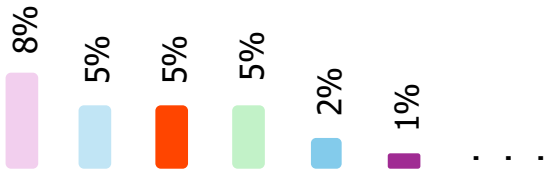
1. Supporting tissue
2. A set of interconnected neurons

2. A set of interconnected neurons

44 1379 902 1041

Connected Neurons

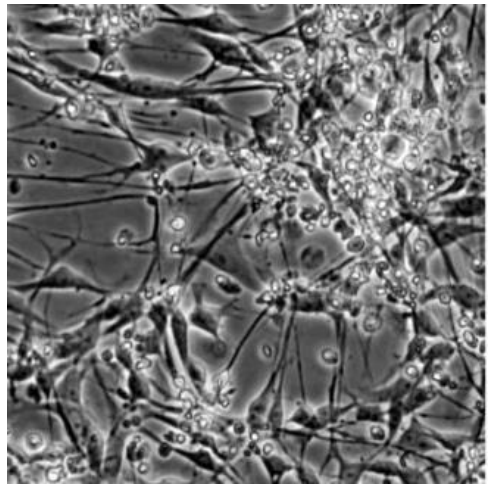





```
probs = model.forward(tokens)
```

2. A set of interconnected neurons

Connected Neurons



=

model.py
~800LOC

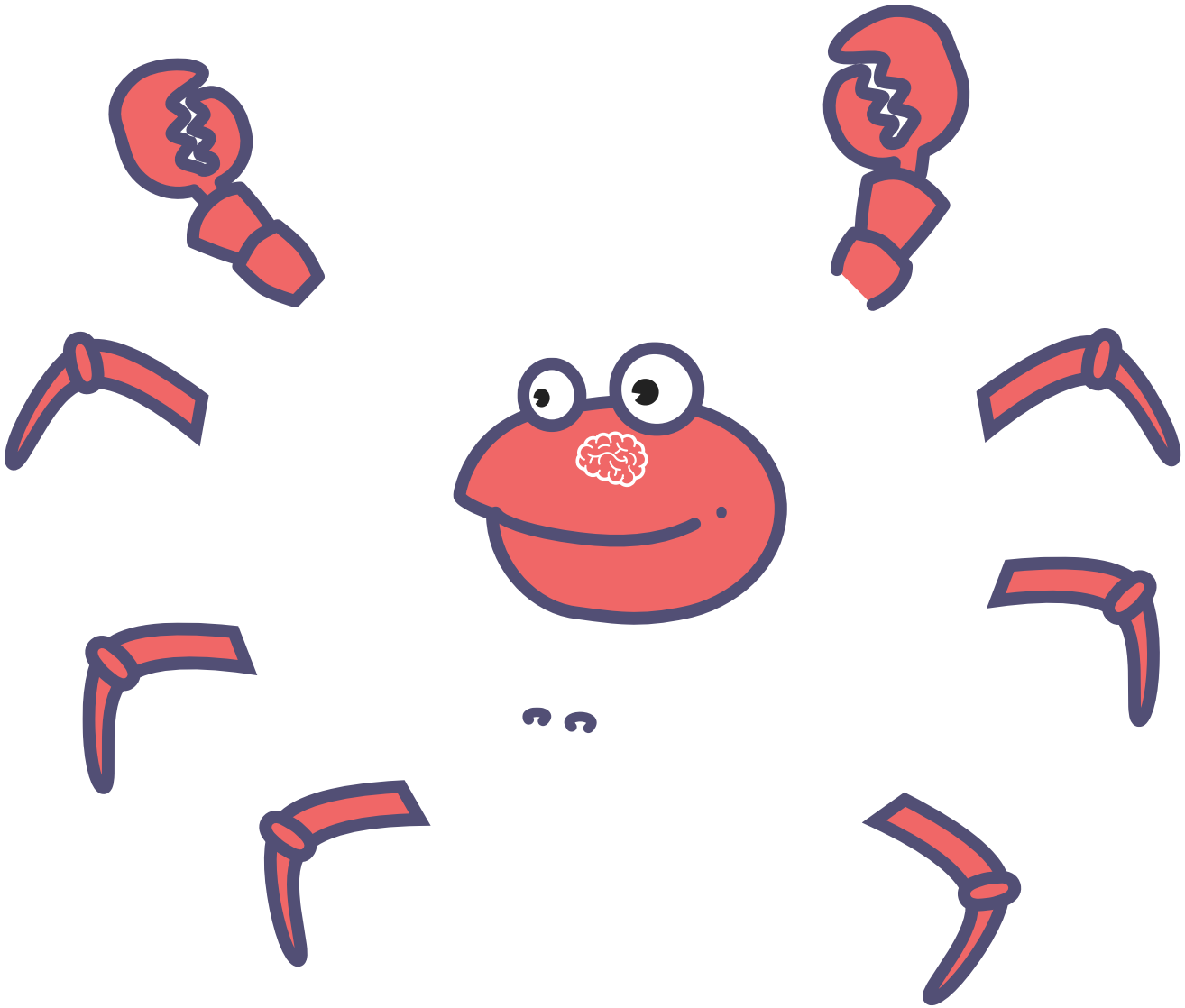
weights.csv
A few hundred billion parameters, requiring a few hundred GB of storage

a **brain**

1. Supporting tissue
2. A set of interconnected neurons

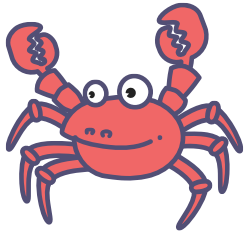
Sidenote

Brains usually live in the cloud



Feels better ☺

Can you check whether we have already saved the pancake recipes?

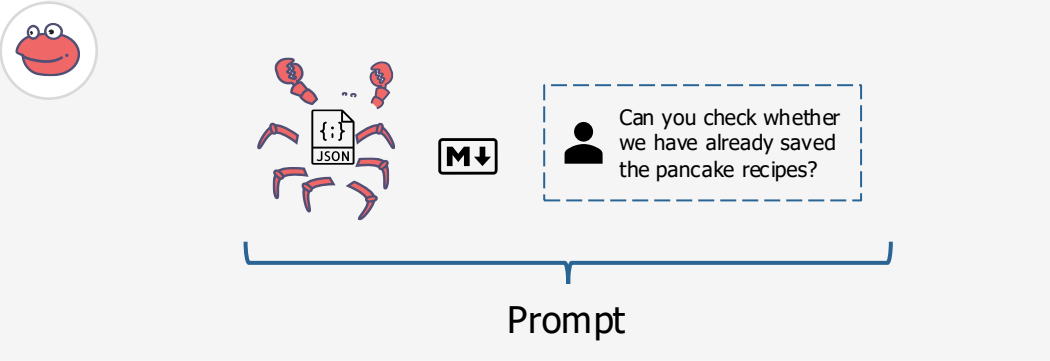


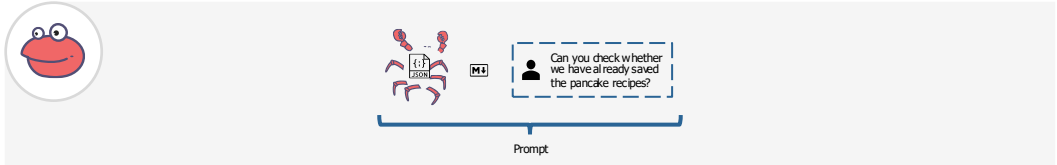
Let me check the current folder

LS(.)

Got it! Yes, we have already saved them to `/pancake-Recipes.md`

Want me to give you step by step instructions for any of the recipes?


JSON
Can you check whether we have already saved the pancake recipes?
Prompt

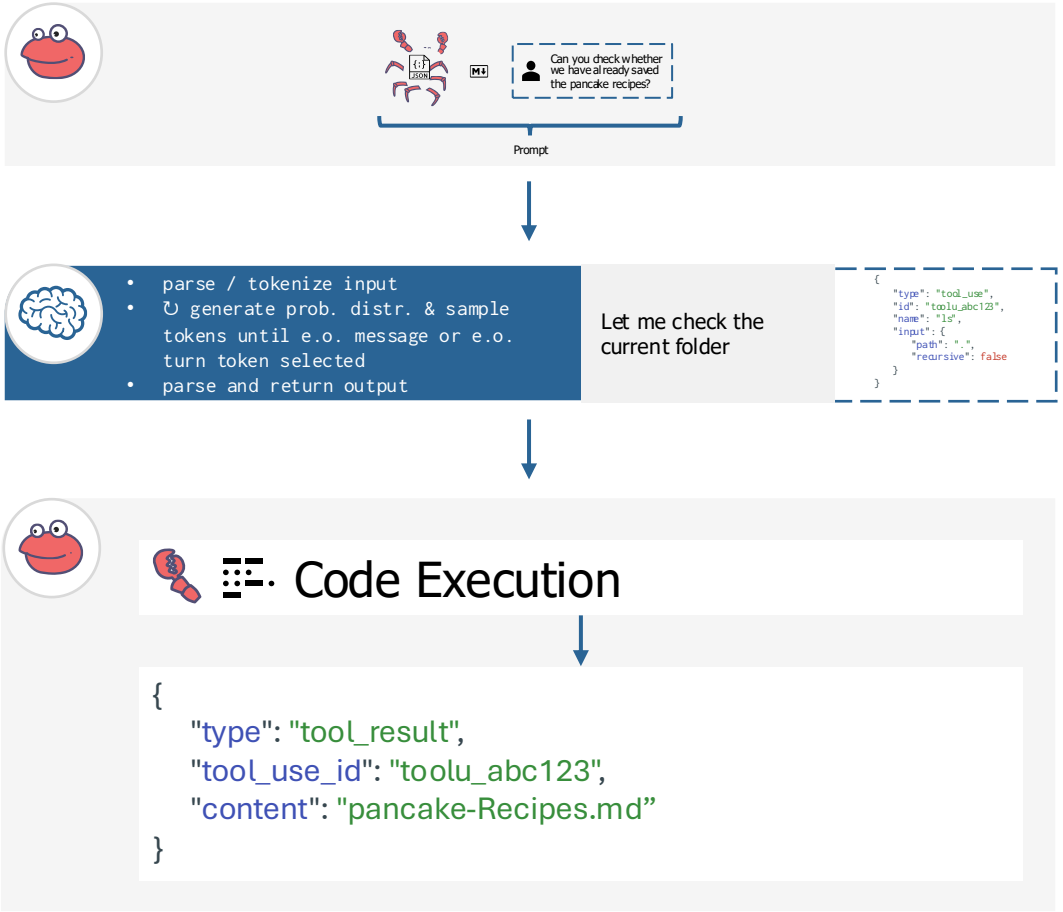


- parse / tokenize input
- ↻ generate prob. distr. & sample tokens until e.o. message or e.o. turn token selected
- parse and return output

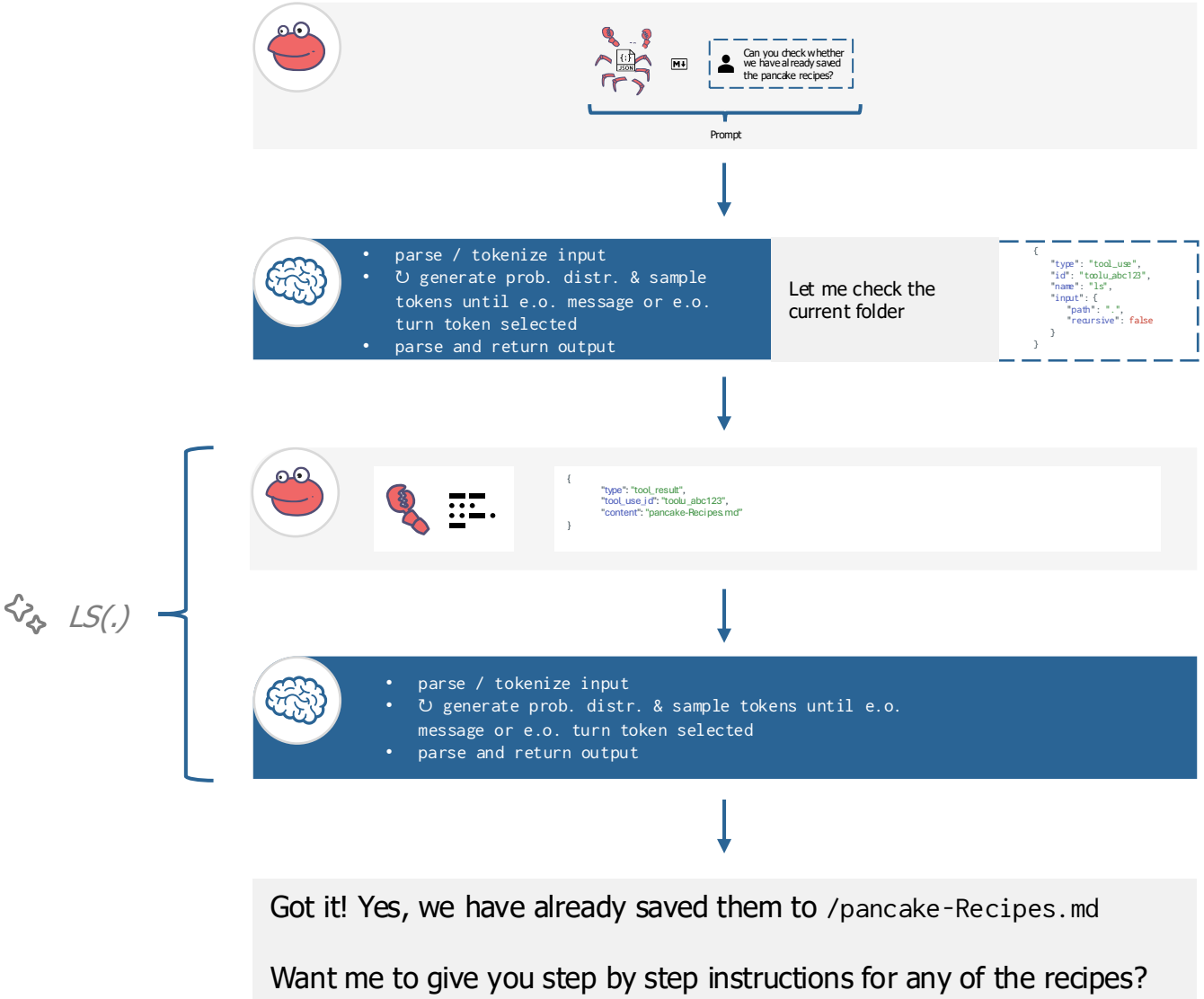
Let me check the current folder

```
{
 "type": "tool_use",
 "id": "toolu_abc123",
 "name": "ls",
 "input": {
  "path": ".",
  "recursive": false
 }
}
```

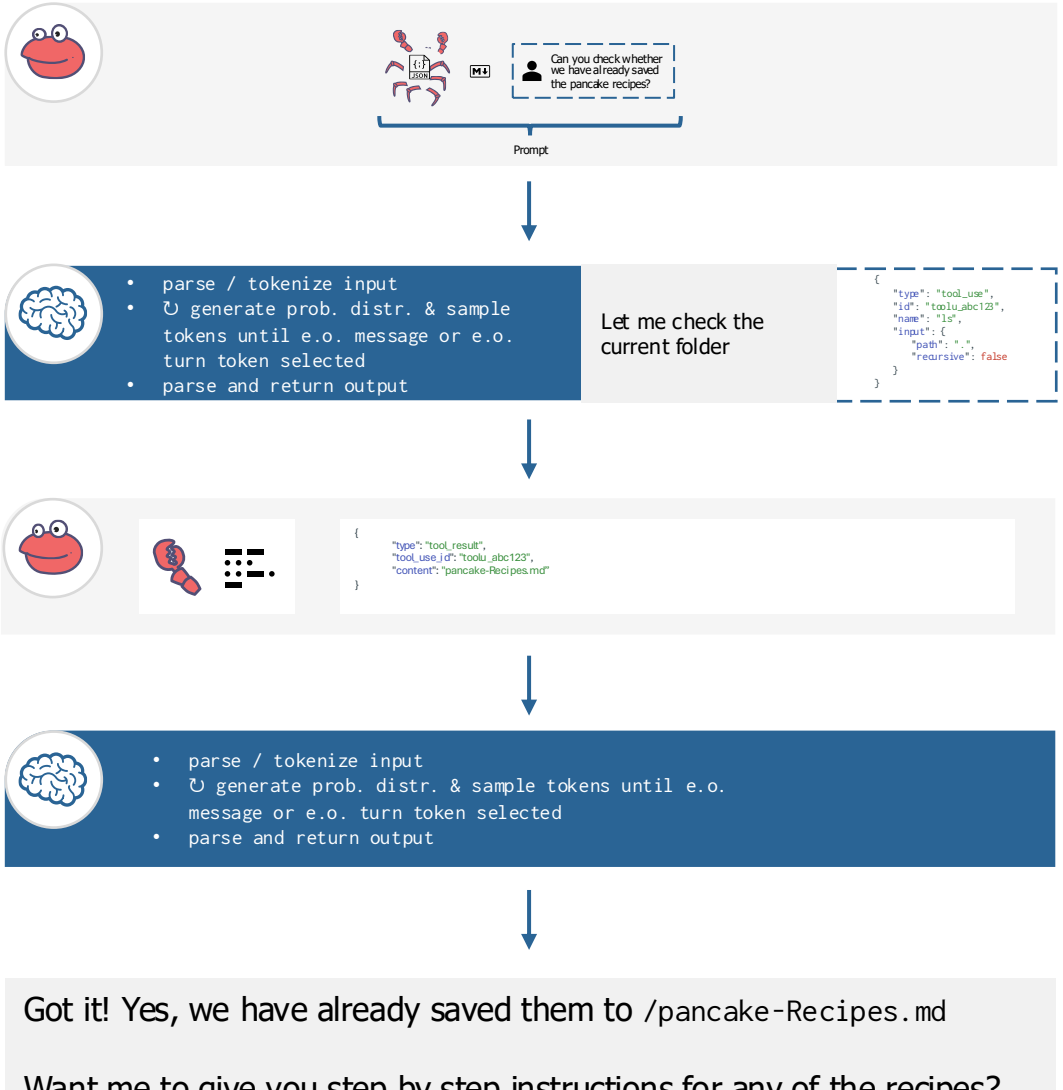
AGENTIC SYSTEMS | JUSTANOTHERCLAW > RECAP



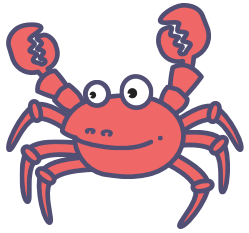
AGENTIC SYSTEMS | JUSTANOTHERCLAW > RECAP



AGENTIC SYSTEMS | JUSTANOTHERCLAW > RECAP



Can you check whether we have already saved the pancake recipes?



Let me check the current folder

LS(.)

Got it! Yes, we have already saved them to `/pancake-Recipes.md`

Want me to give you step by step instructions for any of the recipes?

Questions?

3

Agentic Systems

Tool (Re)use

MAKING TOOLS REUSABLE

JustAnotherClaw

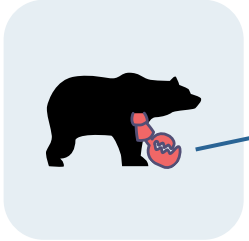
MAKING TOOLS REUSABLE

JustAnotherClaw

AndAnotherClaw

MAKING TOOLS REUSABLE

JustAnotherClaw



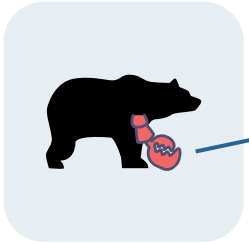
WebFetch

AndAnotherClaw

Portability?

MAKING TOOLS REUSABLE

JustAnotherClaw



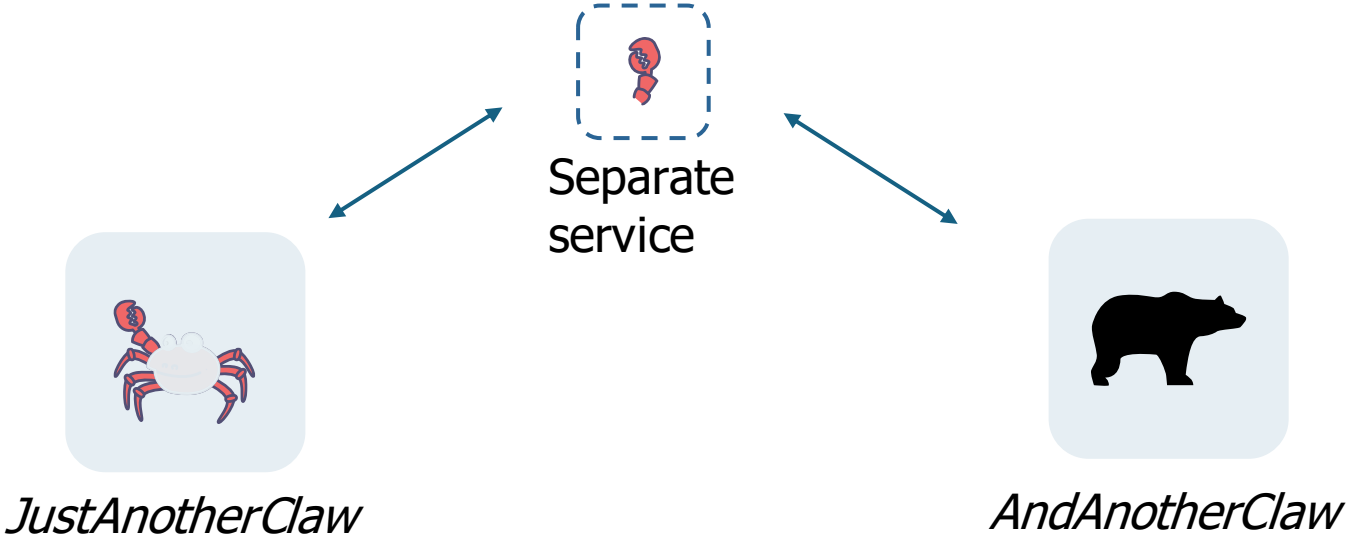
AndAnotherClaw

Super Complex 10k LOC business logic

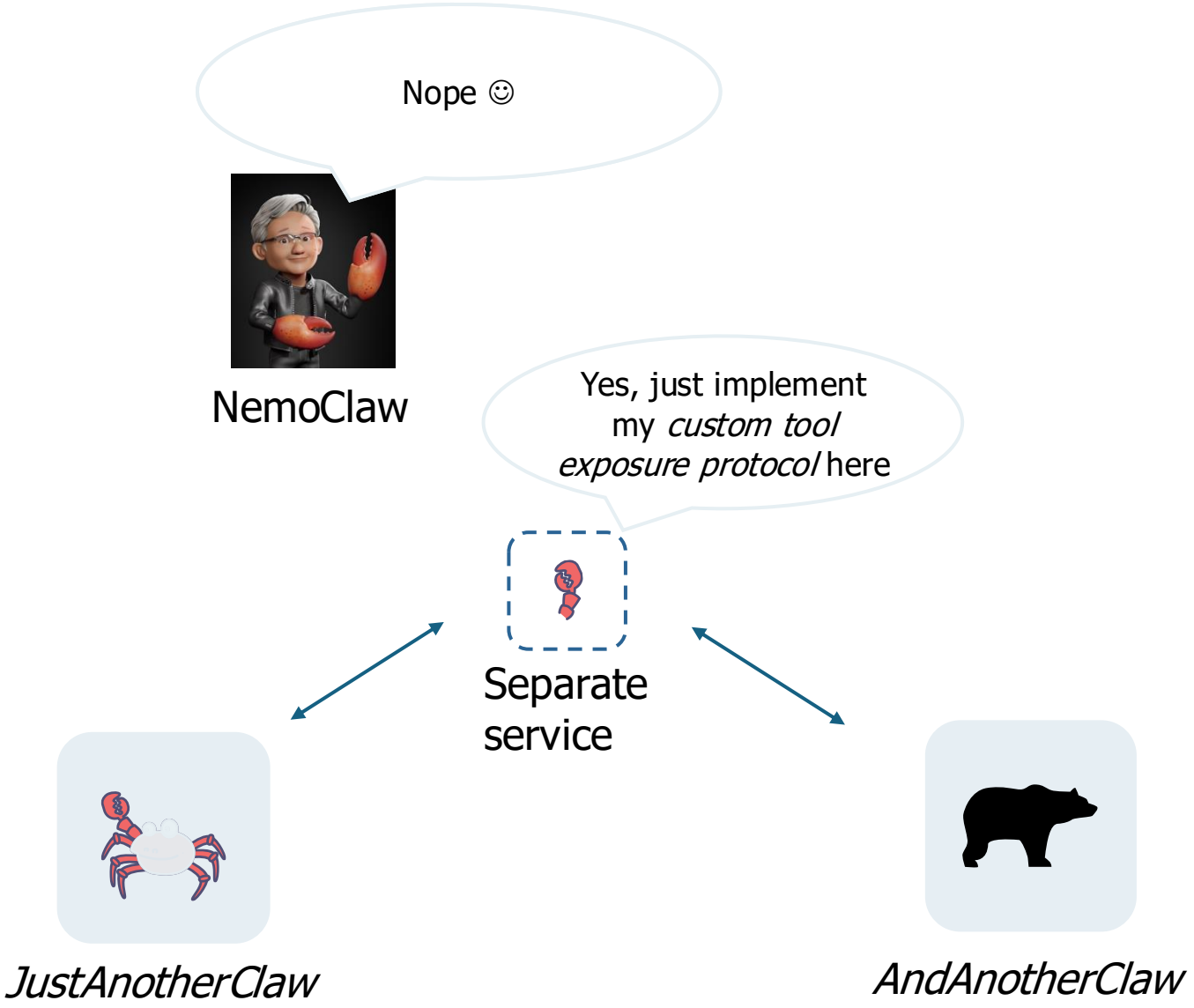
Massive code duplication

Portability?

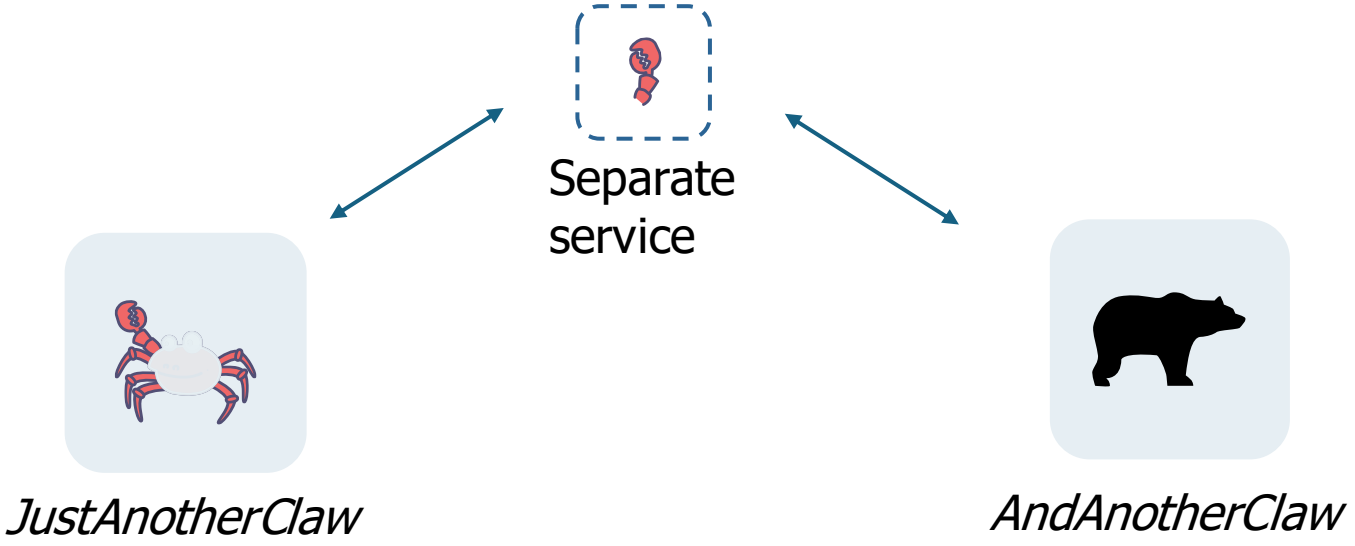
MAKING TOOLS REUSABLE



MAKING TOOLS REUSABLE



MAKING TOOLS REUSABLE



We even call it a "*standard*"!

Why not use our *custom tool exposure protocol*?

Separate service

JustAnotherClaw

AndAnotherClaw

MAKING TOOLS REUSABLE | MCP

Yes, that would be great!

NemoClaw

We even call it a standard!

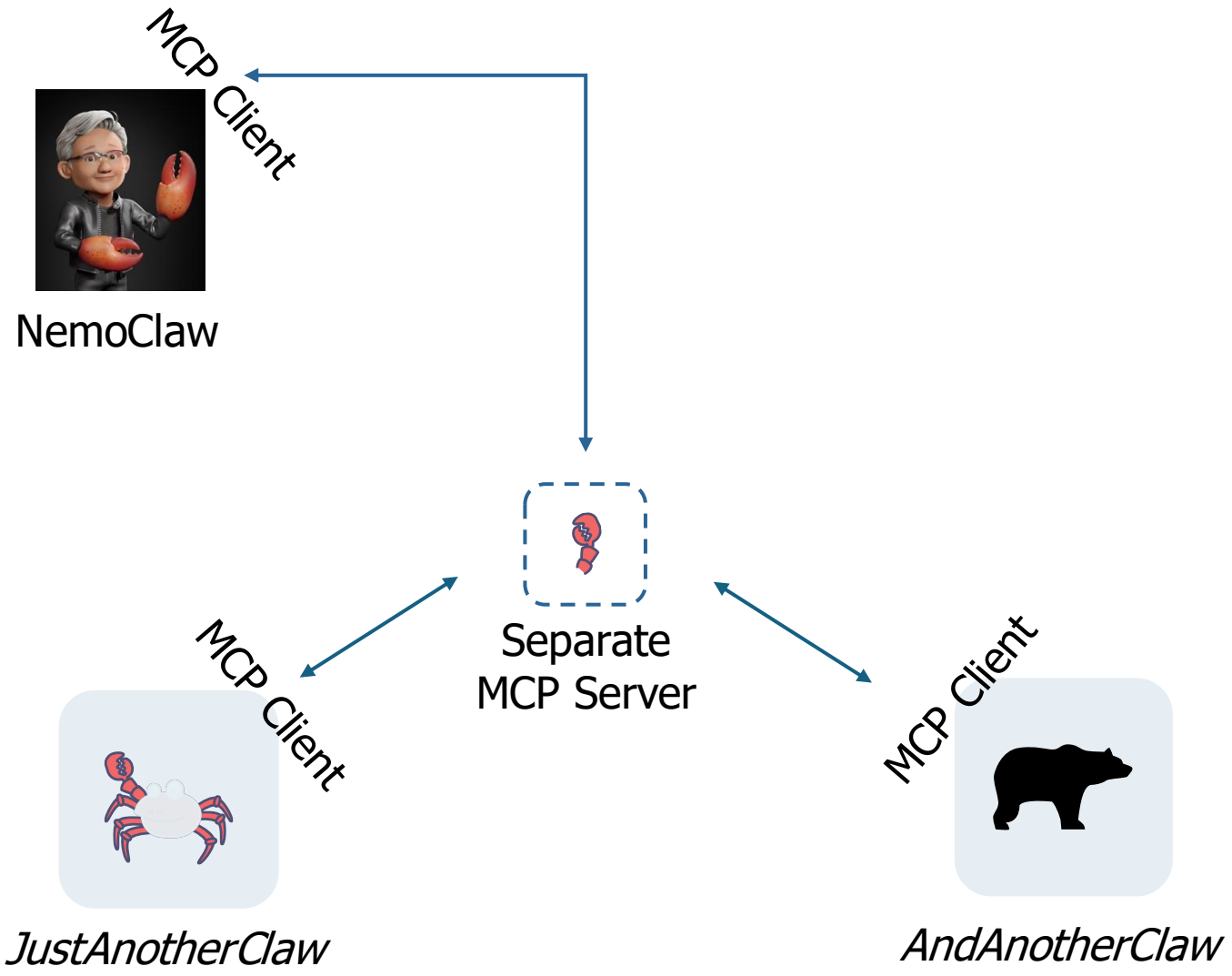
Why not use our *custom tool exposure protocol*?

Separate service

JustAnotherClaw

AndAnotherClaw

MAKING TOOLS REUSABLE | MCP



MAKING TOOLS REUSABLE | MCP

Separate
MCP Server

```
from mcp.server.fastmcp import FastMCP
import urllib.request

mcp = FastMCP("webfetch”)

@mcp.tool()
def web_fetch(url: str) -> str:
  """Fetch the content of a URL and return it as text.""”
  response = urllib.request.urlopen(url)
  return response.read().decode("utf-8")[:5000]

mcp.run()
```

MAKING TOOLS REUSABLE | MCP

NemoClaw

```python
import anthropic
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

# Connect to remote MCP server
async with streamablehttp_client("https://tools.yourdomain.com/webfetchmcp") as
(read, write):
  async with ClientSession(read, write) as session:
    await session.initialize()

    mcp_tools = await session.list_tools() # Discover tools from MCP server

    # Convert to Anthropic tool format
    tools = [{ "name": t.name, "description": t.description,
               "input_schema": t.inputSchema }
           for t in mcp_tools]

    # Call the brain with MCP tools available
    client = anthropic.Anthropic()
    messages = [{"role": "user", "content": "Fetch https://example.com"}]
    response = client.messages.create(
           model="claude-sonnet-4-20250514",
           messages=messages, tools=tools, max_tokens=1024 )

    # Brain returned a tool_use block — execute it via MCP
    for block in response.content:
      if block.type == "tool_use":
        result = await session.call_tool(block.name, block.input)
```

MAKING TOOLS REUSABLE | MCP

Complexity

Context Pollution

Why exactly do we need that?*

```python
import anthropic
from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

# Connect to remote MCP server
async with streamablehttp_client("https://tools.yourdomain.com/webfetchmcp") as
(read, write):
  async with ClientSession(read, write) as session:
    await session.initialize()

    mcp_tools = await session.list_tools() # Discover tools from MCP server

    # Convert to Anthropic tool format
    tools = [{ "name": t.name, "description": t.description,
               "input_schema": t.inputSchema }
           for t in mcp_tools]

    # Call the brain with MCP tools available
    client = anthropic.Anthropic()
    messages = [{"role": "user", "content": "Fetch https://example.com"}]
    response = client.messages.create(
           model="claude-sonnet-4-20250514",
           messages=messages, tools=tools, max_tokens=1024 )

    # Brain returned a tool_use block — execute it via MCP
    for block in response.content:
      if block.type == "tool_use":
        result = await session.call_tool(block.name, block.input)
```

*Reading Suggestion: https://mariozechner.at/posts/2025-11-02-what-if-you-dont-need-mcp/

MAKING TOOLS REUSABLE | TRADITIONAL API

Find me recipe for pancakes?

Sure, let me check what you have in your fridge

calling Fridge API

Got it! Let's search the internet for some inspiration

WebSearch(„Pancake Recipes with milk, […]")

Great! I'll write my top 3 picks into a markdown file.

Write(./pancake-Recipes.md)

Want me to give you step by step instructions for any of the recipes?

MAKING TOOLS REUSABLE | TRADITIONAL API

Find me recipe for pancakes?

But I told you to always use the *https://api.pancakerecipes.com* API first then find the best one, then check my fridge using the fridge cli tool, then use the Billa shopping API to buy what's missing.

Sure, let me check what you have in your fridge

calling Fridge API

Got it! Let's search the internet for some inspiration

WebSearch(„Pancake Recipes with milk, […]")

Great! I'll write my top 3 picks into a markdown file.

Write(./pancake-Recipes.md)

Want me to give you step by step instructions for any of the recipes?

Oops, forgot that

System Prompt

Whenever I ask for pancake recipes use the *https://api.pancakerecipes.com* API first then find the best one, then check my fridge using the fridge cli tool, then use the Billa shopping API to buy what's missing.

Find me recipe for pancakes?

Context Pollution

Sure, let me check the API first

calling Pancake Recipe API

Got it! Let's check the fridge API now

calling Fridge API

Great! Now let's call the Billa API

calling Billa API

Here is your recipe and shopping list: . . .

MAKING TOOLS REUSABLE | TRADITIONAL API + SKILLS

System Prompt

Load this instructions when asked about recipes: /recipes

Find me recipe for pancakes?

Sure, let me load the recipe skill

LoadSkill(recipe)

Okay, let me check the API first

calling Pancake Recipe API

Got it! Let's check the fridge API now

calling Fridge API

Great! Now let's call the Billa API

calling Billa API

Here is your recipe and shopping list: . . .

skills/recipes/SKILL.md

Whenever I ask for pancake recipes use the *https://api.pancakerecipes.com* API first then find the best one, then check my fridge using the fridge cli tool, then use the Billa shopping API to buy what's missing.

MAKING TOOLS REUSABLE | TRADITIONAL API + SKILLS

"Agent Skills are a lightweight, open format for extending AI agent capabilities with specialized knowledge and workflows."

https://agentskills.io/what-are-skills

4

Agentic Systems – A brief look into

Context Management

BRIEF LOOK INTO CONTEXT MANAGEMENT

Ultimately, the LLM still only receives a single piece of text and produces a single probability distribution based on it.

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a general assistant.
You have access to these tools:
{"name": "ls", "description": "List files", "input_schema": {...}}
<|eot_id|><|start_header_id|>user<|end_header_id|>
Can you check whether we have already saved the pancake recipes
<eot_id|><|start_header_id|>assistant<|end_header_id|>
Let me check.
<tool_call>
{"name": "ls", "input": {"path": ”."}}
</tool_call><|eot_id|><|start_header_id|>user<|end_header_id|>
[Tool result for toolu_abc]: pancake-Recipes.md<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Context Window Limit

BRIEF LOOK INTO CONTEXT MANAGEMENT

1 Compacting

```
if token_count(messages) > 0.8 * CONTEXT_LIMIT:
    summary = call_brain("Summarize this conversation:" + str(messages))
    messages = [
        system_prompt,
        {"role": "user", "content": f"[SUMMARY]\n{summary}\n\nContinue."}
    ]
```

2 Memory

- Simple “memory-cli” as a set of simple read/write to .md (or alternative format) instructions
- When and how to search / persist information is a design decision injected in the system prompt and / or the agentic loop itself (e.g. on session start, on compacting, etc.)

2 Sub-Agents

= another “text in - text out, in a loop and with side effects”-process with the main agent loop

ADDITIONAL RESOURCES

Open Claw:

- https://github.com/badlogic/pi-mono (backbone behind openclaw)
- https://mariozechner.at/posts/2025-11-30-pi-coding-agent/ (retrospective on building pi-mono)
- https://github.com/openclaw/openclaw

Paper that formally decompose agent architecture:

- https://arxiv.org/abs/2603.05344

- A bit more complex examples of coding agent harnesses:
  - https://github.com/charmbracelet/crush
  - https://github.com/Aider-AI/aider